

METAHEURÍSTICAS DE BUSCA LOCAL

DCE770 - Heurísticas e Metaheurísticas

Atualizado em: 11 de setembro de 2025

Iago Carvalho

Departamento de Ciência da Computação



ALGORITMO, HEURÍSTICA E META-HEURÍSTICA

Um algoritmo computa a resposta exata para um problema específico

Uma heurística computa uma solução aproximada para um problema específico

Uma meta-heurística é um *framework* para construção de heurísticas

- Não resolvem um problema específico
- Possibilita criar heurísticas para diversos problemas
- São extremamente generalizáveis

A partir de agora, esse curso focará em **meta-heurísticas**!

Problemas NP-Completo possuem um número exponencial de soluções

- Problema $\#P$ -Completo
- Impraticável listar todas elas

Meta-heurísticas exploram um subconjunto destas soluções de forma não-aleatória

Uma meta-heurística é um *framework*, um "guia", sobre como explorar esse subconjunto de soluções

- Quanto mais *eficaz* e mais *eficiente* for esta amostragem **melhor** é a heurística resultante

CARACTERÍSTICAS DE META-HEURÍSTICAS

Simplicidade

- São baseadas em princípios claros

Generalidade

- Podem ser facilmente generalizadas para diversos problemas

Eficácia

- Produzem soluções de boa qualidade

Eficiência

- Baixo custo computacional

Diversificação

- Como ela realiza a busca
- Ato de explorar uma grande área do espaço de buscas

Intensificação

- Buscas realizadas em soluções próximas a outras
- No geral, tende-se a intensificar a busca próximo a soluções de boa qualidade

Na aula passada entendemos o conceito de vizinhança e busca local

- Uma busca local é um processo de intensificação

Além disso, também estudamos o algoritmo de Hill Climbing

- Uma primeira heurística de busca local
- Utiliza um único esquema de vizinhança
- Capaz de atingir um ótimo local
- Incapaz de atingir múltiplos ótimos locais
- Provavelmente incapaz de atingir o ótimo global

Existem diversas abordagens mais sofisticadas na literatura

MULTI-START

MULTI-START

O algoritmo de **Multi-start** é capaz de atingir múltiplos ótimos locais

Ele roda diversas vezes o algoritmo de Hill Climbing

- Cada execução utiliza uma solução inicial diferente
 - Heurística construtiva gulosa semi-aleatória
- Múltiplas iterações funcionam como um mecanismo de diversificação

O algoritmo de Multi-start é executado até que um critério de parada seja atingido

- Tempo de execução máximo
- Número de iterações sem melhora da função objetivo

Algoritmo 1: Multi-start

Entrada: $N(s)$ // Estrutura de vizinhança

```
1 enquanto critério de parada não atingido faça
2    $s \leftarrow$  solução inicial gulosa semi-aleatória
3    $s' \leftarrow \text{HillClimbing}(s, N(s))$  // Realiza busca local
4    $s'' \leftarrow \text{melhor}(s', s'')$  // Salva a melhor solução
5 retorne  $s''$ 
```

O Multi-start é capaz de atingir múltiplos ótimos locais

- Um ótimo local em cada iteração

Entretanto, ele ainda não é capaz de *escapar* destes ótimos locais

- Utiliza um único esquema de vizinhança
- Todos seus operadores são determinísticos

VARIABLE NEIGHBORHOOD SEARCH
E

VARIABLE NEIGHBORHOOD DESCENT

VARIABLE NEIGHBORHOOD DESCENT

O algoritmo VND foi proposto por Mladenovic e Hansen (1997)

- Mladenović N, Hansen P. Variable neighborhood search. Computers & Operations Research. 1997 Nov 1;24(11):1097-100. [▶ Link](#)
- Uma melhor e mais completa referência pode ser encontrada em [▶ Link](#)

Explora diversas estruturas de vizinhança de forma iterativa

- Diferentes vizinhanças induzem a diferentes ótimos locais
- Explorar diversas vizinhanças pode levar a ótimos locais de melhor qualidade

VND - PRINCÍPIOS BÁSICOS

O VND opera com um conjunto pré-definido e ordenado de estruturas de vizinhança

- Denota-se como $N_i(s)$ a i -ésima vizinhança do VND
- Conjunto de vizinhanças definido como $N_k(s)$
- Vizinhanças costumam ser ordenadas por complexidade
 - Da menos complexa para a mais complexa

Preocupação com o tempo de execução

- Vizinhanças menos complexas são exploradas primeiro
- Vizinhanças complexas são pouco exploradas
 - Utilizadas como *último recurso* para sair de um ótimo local
- Em geral, quando há uma melhora, retorna-se uma vizinhança de menor complexidade

O VND pode ser sumarizado em 4 passos

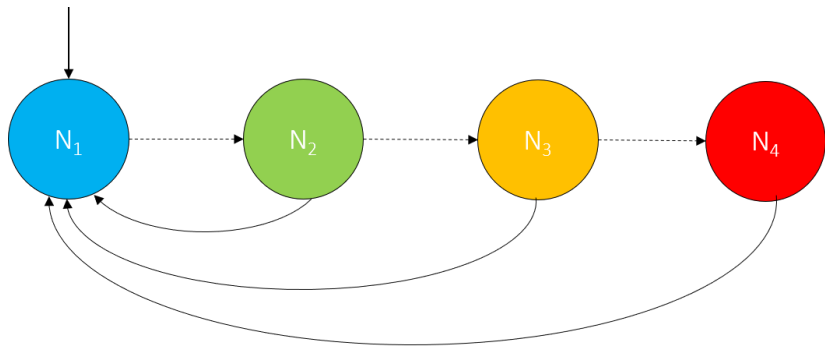
1. **Inicialização:** Obtém uma solução inicial s e define o índice de vizinhança $i = 1$
2. **Busca:** Explora a vizinhança $N_i(s)$ em busca de uma solução s' melhor que s
3. **Movimento e reinício:** Se s' é encontrada, então $s \leftarrow s'$ e $i \leftarrow i$. Caso contrário, $i \leftarrow i + 1$
4. **Término:** O algoritmo termina quando algum critério de parada é atingido
 - Tempo máximo de execução
 - $i > r$, onde $r = |N_k(s)|$

Por definição, a solução final s será considerada um ótimo local em relação a todas as r estruturas de vizinhança exploradas

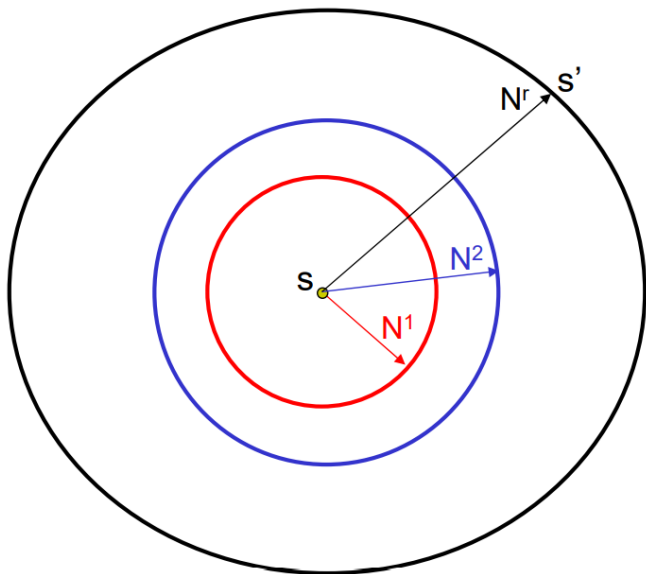
Algoritmo 2: Variable Neighborhood Descent

```
Entrada:  $N_k(s)$  // Estruturas de vizinhança
Entrada:  $s$  // Solução inicial
Entrada:  $i \leftarrow 1$  // Contador de vizinhanças
1  $r \leftarrow |N_k(s)|$  // Número de vizinhanças
2 enquanto  $i \leq r$  faça
3    $s' \leftarrow \text{HillClimbing}(s, N_i(s))$  // Busca local
4   se  $s'$  é melhor que  $s$  então
5      $s \leftarrow s'$  // Salva a melhor solução
6      $i \leftarrow 1$  // Reinicia as vizinhanças
7   caso contrário
8      $i \leftarrow i + 1$  // Incrementa a vizinhança
9 retorne  $s$ 
```

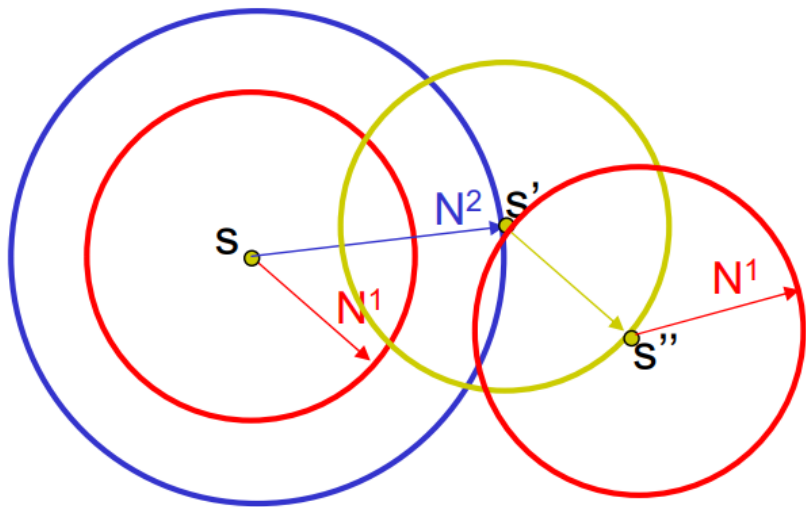
VND - TROCA DE VIZINHANÇAS



VND - VIZINHANÇAS MAIS COMPLEXAS



VND - TROCA DE VIZINHANÇAS



Existem diversas variações do VND que trocam as vizinhanças de maneiras diferentes

Algoritmo 3: Variable Neighborhood Descent generalizado

Entrada: $N_k(s)$ // Estruturas de vizinhança

Entrada: s // Solução inicial

```
1 enquanto critério de parada não atingido faça
2    $s' \leftarrow \text{HillClimbing}(s, N_i(s))$ 
3    $s \leftarrow \text{melhor}(s, s')$  // Obtém o melhor entre  $s$  e  $s'$ 
4   AlteraVizinhança( $i, N_k(s)$ )
5 retorne  $s$ 
```

VARIAÇÕES DO VND

As diversas variações do VND implementam a função *AlteraVizinhança* de forma diferente

- Linha 4 do pseudo-código anterior

Alguns exemplos

1. *Pipe Neighborhood change*

- Se houver melhora em uma vizinhança, não realiza a troca
- Caso contrário, utiliza a vizinhança seguinte

2. *Cyclic neighborhood change*

- Utiliza a vizinhança seguinte independente do resultado de melhora ou não

3. *Random neighborhood change*

- Variação das três anteriores
- Entretanto, não há uma ordem definida para troca
- Trocas realizadas de forma aleatória

CRÍTICA AO VND

A principal força do VND reside em sua capacidade de **intensificação**

- Realizar buscas ao redor de soluções construídas por heurísticas construtivas
- Consegue obter um ótimo local de boa qualidade ao combinar diversos esquemas de vizinhança

Entretanto, ele é altamente dependente da solução inicial dado pela heurística construtiva

Ele não é capaz de escapar de ótimos locais ou fazer uma exploração efetiva do espaço de soluções

Por causa disso, raramente vemos um VND sozinho

- Normalmente ele é utilizado como mecanismo de busca local

O algoritmo VNS foi proposto junto do VND

- Mesmas referências

A principal diferença do VND para o VNS é o *shake*

- Mudança aleatória da solução corrente
- Similar a um operador de mutação em algoritmos evolutivos

Com este operador, torna-se possível escapar de ótimos globais e realizar uma melhor busca no espaço de soluções

O VNS pode ser sumarizado em 5 passos

1. **Inicialização:** Obtém uma solução inicial s utilizando uma heurística construtiva
2. **Shake:** Uma solução $s' \in N(s)$ é selecionada aleatoriamente
3. **Busca:** Realiza uma busca local em torno de s' até encontrar um ótimo local, representado por uma solução s''
4. **Movimento e reinício:** Se uma solução aprimorante s'' é encontrada, então $s \leftarrow s''$. Logo após, voltamos para o passo 2
5. **Término:** O algoritmo termina quando algum critério de parada é atingido
 - Tempo máximo de execução
 - Numero máximo de iterações

Algoritmo 4: Variable Neighborhood Search

Entrada: $N_k(s)$ // Estruturas de vizinhança

Entrada: s // Solução inicial

```
1 enquanto critério de parada não atingido faça
2    $s' \leftarrow \text{shake}(s, N_k(s))$  //  $s' \leftarrow$  vizinho aleatório de  $s$ 
3    $s'' \leftarrow \text{buscaLocal}(s', N_k(s))$ 
4   se  $s''$  é melhor que  $s$  então
5      $s \leftarrow s''$ 
6 retorne  $s$ 
```

O VNS pode implementar qualquer algoritmo como esquema de busca local

- *Hill Climbing*
- *Multi-start*
- VND
- ...

A qualidade das soluções do VNS dependem do esquema de vizinhanças definido e do algoritmo de busca local implementado

VNS - VARIAÇÕES

Reduced VNS (RVNS)

- Não existe busca local (linha 3 do pseudo-código 4)
- Escolhe um vizinho de forma aleatória na vizinhança corrente
 - Caso exista melhora, move-se para este vizinho e continua o processo; Caso contrário, troca-se de vizinhança
- Critério de parada idêntico ao do VNS

Skewed VNS (SVNS)

- Aceita soluções de pior custo durante o processo de busca local
- O objetivo é explorar o espaço de buscas de forma mais ampla

Smart VNS

- Aumenta o tamanho do *shake* iterativamente
- Inicialmente, favorece a intensificação
- Após, favorece a exploração do espaço de buscas
- Caso haja melhora, o tamanho do *shake* é novamente reduzido

GREEDY RANDOMIZED ADAPTIVE
SEARCH PROCEDURE
(GRASP)

GREEDY RANDOMIZED ADAPTIVE SEARCH PROCEDURE

O algoritmo GRASP foi proposto por Fao e Resende (1995)

- Feo, T. A. e Resende, M. G. C. Greedy Randomized Adaptive Search Procedures. *Journal of Global Optimization*, 6:109-133, 1995 [▶ Link](#)

O algoritmo é iterativo, e cada iteração é composta por duas fases

1. Construção
 - Constroi uma solução de forma gulosa semi-aleatória
2. Refinamento
 - Utiliza buscas locais para melhorar a solução inicial obtida

De certa forma, ele lembra muito o *Multi-start*

GRASP - FASE DE CONSTRUÇÃO

A fase de construção é parcialmente gulosa e parcialmente aleatória

A parte gulosa é necessária para

- Gerar soluções de boa qualidade
- Acelerar a convergência a ótimos locais (e possivelmente globais)

Necessário inserir aleatoriedade devido a natureza iterativa do algoritmo

- A cada iteração, deve-se gerar uma solução diferente
- Desta forma, é possível explorar, de forma efetiva, o espaço de soluções

GRASP - FASE DE REFINAMENTO

Aplica um algoritmo de busca local para refinar a solução obtida anteriormente

Inicialmente, utiliza um algoritmo *Hill Climbing* com um único esquema de vizinhança

Entretanto, outras estratégias mais inteligentes podem ser utilizadas

- VND
- Multi-start
- ILS
- Busca Tabu
- ...

Veremos algumas destas estratégias mais a frente

Algoritmo 5: GRASP

Entrada: $N_k(s)$ // Estruturas de vizinhança

Entrada: α // Grau de aleatoriedade

1 **enquanto** *critério de parada não atingido* **faça**

2 $s \leftarrow \text{heuristicaGulosa}(\alpha)$

3 $s' \leftarrow \text{buscaLocal}(s, N_k(s))$ // Realiza busca local

4 $s'' \leftarrow \text{melhor}(s', s'')$ // Salva a melhor solução

5 **retorne** s''

A maior diferença do GRASP para o Multi-start está no parâmetro α

- Quanto maior o valor de α , mais aleatória é a solução
- Quanto menor seu valor, mais gulosa é a solução

Como dito anteriormente, ele pode implementar qualquer algoritmo na fase de busca local

Além dos diversos algoritmos que podem ser utilizados na etapa de busca local do GRASP, também existem variações interessantes sobre como criar as soluções gulosas semi-aleatórias

A primeira utiliza uma *lista de candidatos restrita* (LCR)

A segunda faz com que α seja ajustado dinamicamente durante o GRASP

A terceira combina as duas anteriores em um único algoritmo

A heurística gulosa torna-se *menos gulosa*(?)

Ao invés de tomar uma decisão míope, ele gera um conjunto de elementos (LCR) que podem ser adicionados

- Elementos com valor de proporção α do melhor elemento possível

Então, a decisão torna-se gulosa semi-aleatória

- Um item aleatório dentro da LCR é escolhido

O valor de α é escolhido aleatoriamente a cada iteração do GRASP

Também pode-se realizar uma adaptação dinâmica do valor de α

1. Seleciona-se um conjunto de valores possíveis para α
2. Definimos uma probabilidade igual de escolha para cada um dos valores anteriores
3. A cada iteração, sorteamos um valor de α seguindo as probabilidades definidas
4. Caso a iteração obtenha uma solução aprimorante, aumentamos a probabilidade do valor de α escolhido
 - Caso contrário, diminuimos sua probabilidade e aumentamos a dos outros valores de α